Casper in the Rho Calculus

Justifications as quotation, equivocation as a join,
and the hypothesis the clique oracle needs

L. G. Meredith
F1R3FLY.io

October 18, 2026 – Draft 1

Abstract

We give a model of the core of CBC Casper in the rho calculus with group-valued data of *Oriented Processes*. Block construction and execution are abstracted away; what remains is the message structure that Casper's safety argument uses. A block is a quoted process, and its justification pointers are the names that occur in it, so the hash-linked DAG is reflection: content addressing, acyclicity and justification closure hold by construction rather than by appeal to collision resistance. The datum of a block's name is its signer, and the group action places each validator in its own frame, so the validator set is a product of rotated copies of one program, signatures become a discipline on frames, and the protocol is equivariant under relabelling of validators. Equivocation detection becomes a single join on a *slot*: a name that an observer builds from a block's self-justification pointer. Two announcements meet on a slot exactly when the signer has equivocated, so the comparison is performed by the gate of the communication rule and not by an equality test. We prove that the slot criterion coincides with CBC's incomparability criterion and with the implementation's sequence-number criterion, that the detector is sound in every reachable state and complete under fairness, and that honest views are protocol states whose fault weight is bounded by the Byzantine weight. On this footing we prove CBC consensus safety and a clique-oracle theorem for any majoritarian estimator, which includes LMD-GHOST. The clique theorem has a hypothesis that the oracle in `f1r3node-rust` does not check: that a's view of b itself supports the target. We exhibit an asynchronous schedule, with no equivocation and under the default configuration, in which that oracle certifies two sibling blocks, and we give the one-line repair.

Contents

1 Introduction **2**

2 The calculus **3**

3 Blocks as quoted processes **4**
3.1 Blocks 4
3.2 Protocol states and validity 5
3.3 Equivocation 5

4 Validators as frames **6**
4.1 One program, many frames 6
4.2 Signatures as a frame discipline 6
4.3 The system 7

5 The honest validator **7**
5.1 Terms 7
5.2 Observers 8

6 Equivocation as a join **8**

7 Safety **9**
7.1 Views 9
7.2 Consensus safety 10

8 The clique oracle **10**
8.1 Majoritarian estimators 10
8.2 The theorem 11

9 The implementation **12**
9.1 Crosswalk 12
9.2 The oracle omits a hypothesis 12
9.3 A counterexample 13
9.4 Repair 14

10 What the group contributes **14**

11 Further work **14**

1 Introduction

CBC Casper is unusual among consensus protocols in that its safety theorem is almost a tautology once the right objects are in place [4, 5]. A *protocol state* is a set of messages closed under justification. Its *futures* are the larger protocol states whose fault weight stays under a threshold. A property is *safe* at a state when the estimator satisfies it in every future. Two safe properties held by two observers are then consistent, because the union of their states is a common future. All the work is in making the objects real: messages must carry their justifications, so that the union is closed and so that equivocation, the only fault the argument counts, is visible in it.

That is the observation this note starts from. The main idea in Casper is the justification pointer. It is what lets a union of views be formed, what lets an equivocation be seen, and what lets an observer compute a fault weight without trusting anyone. In the rho calculus a pointer to a process is a name, and a name is a quoted process. The calculus therefore already contains the data structure that Casper is built on, and a model of Casper in it has an unusually short distance to travel.

What is modelled. Validators with fixed weights; blocks consisting of a signer, a self-justification, an estimate and a set of justifications; well-formedness and estimate validity, checked hereditarily; reliable asynchronous delivery; honest validators that merge, detect, propose and decide; and Byzantine validators constrained only by unforgeability of signatures.

What is abstracted. Deploys, execution, state hashes, merging of post-states, bonding, slashing economics, epochs and the synchrony constraint. Local computation inside a validator (merging a view, evaluating the estimator, running the oracle) is a deterministic terminating function of quoted data, implementable as a rho process, and taken as one atomic step. The interest of the model is the concurrency *between* validators.

Contributions.

- Blocks as quoted processes (Section 3): the justification DAG is the subterm order on codes, and it is acyclic and content-addressed by construction (Lemma 3.3). Under the well-formedness rules that the implementation enforces, three definitions of equivocation coincide: incomparability under justification, sharing a self-justification, and sharing a height (Proposition 3.8).
- Validators as frames (Section 4): the honest network is a product of rotations of one program, signatures are a frame discipline, all communication is between equal names so the model lives in the conservative fragment of rho$_G$, and the protocol is equivariant (Proposition 4.4).
- Equivocation as a join (Section 6): the slot detector, sound in every reachable state and complete under weak fairness (Theorem 6.1).
- Safety (Section 7): honest views are valid protocol states, honest validators never equivocate, and CBC consensus safety holds in the model (Theorem 7.3).
- The clique oracle (Section 8): a soundness theorem for any majoritarian estimator, including LMD-GHOST (Theorem 8.5), and finality safety (Corollary 8.6).
- The implementation (Section 9): a crosswalk to `f1r3node-rust`, and a counterexample showing that its clique oracle omits a hypothesis of Theorem 8.5, with a repair.

2 The calculus

We work in rho$_G$, the reflective higher-order calculus with data drawn from a group G [2]. We recall only what is used.

A name $@\langle P, d\rangle$ pairs a *code* P with a datum $d \in G$; we write $\nu(@\langle P, d\rangle) = d$. A message $x!(\langle Q, d\rangle)$ ships a pair that is received as the name $@\langle Q, d\rangle$; we write $x!(n)$ for the message that ships the name n intact, and use polyadic sends as the usual sugar. A receive $\texttt{for}(\pi \leftarrow x)P$ binds the variables of the pattern π, which may mention literal data. The group acts on processes by left multiplication of every datum, at every depth of quotation, and is deferred at variables until they are instantiated; $g \cdot P$ is the rotation of P by g. Reduction is equivariant (Theorem 3.4 of [2]), and a receive rotated by g binds what it receives in its own frame, so that a rotated program uses a received name exactly as it arrived (Lemma 3.3 of [2]).

We add two standard prefixes: the *join* $\texttt{for}(\pi_1 \leftarrow x_1 \,\&\, \pi_2 \leftarrow x_2)P$, which consumes one matching message on each of its channels in one step, and the *persistent* receive $\texttt{for}(\pi \Leftarrow x)P$, which is not consumed. Equivariance extends to both verbatim, since its proof is a per-prefix calculation with the same coefficients. The chaos constant and summation of rho_G play no part in the model, and Section 4 shows that the oriented coefficients of the communication rule are always the identity. The group is used for its *action*: for frames.

Local computations. We write $\texttt{let}\ y = f(x)\ \texttt{in}\ P$ for a step whose effect is to continue as $P\{f(x)/y\}$, where f is a total function on codes defined by structural recursion. Every such f below is implementable as a deterministic terminating rho process whose internal steps are confluent, so treating it as atomic changes no observable behaviour; it corresponds to a validator holding its block-processing lock.

3 Blocks as quoted processes

3.1 Blocks

Fix a finite set $V \subseteq G$ of validators with weights $w : V \to \mathbb{N}$, $W = \sum_{v \in V} w(v)$, and for $U \subseteq V$ write $w(U) = \sum_{u \in U} w(u)$. Fix base names slf, est, jst, anc whose codes are pairwise non-congruent and each mention exactly one name; for $v \in G$ write $x_v = v \cdot x$. Estimates are codes drawn from a fixed set.

Definition 3.1 (Blocks)**.** The *anchor* of v is $\alpha_v = @\langle anc_v!(\mathbf{0}), v\rangle$. A *block* by v with self-justification p, estimate E and justifications J (a finite set of names) is the name

$$\lfloor v;\, p,\, E,\, J\rfloor \;=\; @\Big\langle\, slf_v!(p) \;\mid\; est_v!(E) \;\mid\; \textstyle\prod_{n\in J} jst_v!(n)\,,\; v\Big\rangle.$$

For a block $m = \lfloor v;\, p,\, E,\, J\rfloor$ write $\mathrm{sdr}(m) = \nu(m) = v$, $\mathrm{self}(m) = p$, $\mathrm{est}(m) = E$ and $J(m) = \{p\} \cup J$.

The block's structure is written in its signer's frame and its contents are other names, intact. The datum of the block's name is the signer, and Section 4 explains why an observer can read the frame off the datum. Note what is *not* in a block: no hash. The name of the block is the block.

Definition 3.2 (Closure)**.** $J^*(\alpha_v) = \emptyset$ and $J^*(m) = J(m) \cup \bigcup_{n \in J(m)} J^*(n)$. Write $n \le m$ when $n = m$ or $n \in J^*(m)$.

Lemma 3.3 (Reflection is a DAG)**.** *(i) If $n \in J(m)$ then n is a proper subterm of the code of m, so $J^*(m)$ is finite and $m \notin J^*(m)$; the order $\le$ is a well-founded partial order.*

(ii) $J^*(m)$ is a function of m alone, and two blocks are equal exactly when their codes are congruent and their signers agree.

Proof. (i) A justification occurs quoted inside a message of the block's code, so its code is strictly smaller; induct on the size of codes. (ii) Names are equal when their codes are congruent and their data equal; J^* is defined by recursion on the code. □

In an implementation, acyclicity and content addressing are consequences of the collision resistance of a hash. Here they are structural facts. The price is that a block carries its whole history; Section 9 returns to this.

3.2 Protocol states and validity

Definition 3.4 (Protocol state)**.** A *protocol state* is a finite set σ of blocks and anchors that contains every anchor and is closed under J. For $v \in V$ write σ_v for the blocks of σ signed by v, anchors excluded.

An *estimator* is a map Est from protocol states to non-empty sets of estimates, depending only on the latest messages of non-equivocating validators (Definition 3.7). Following [4], nothing in Sections 3–7 depends on which estimator is used.

Definition 3.5 (Validity)**.** A block m by v is *well formed* if

(W1) $\mathrm{self}(m)$ is a block by v or is α_v, and

(W2) every block by v in $J^*(m)$ is in $\{\mathrm{self}(m)\} \cup J^*(\mathrm{self}(m))$.

It is *valid* if every block in $J^*(m) \cup \{m\}$ is well formed and satisfies

(V) $\mathrm{est}(n) \in \mathrm{Est}(J^*(n) \cup A)$, with A the set of anchors.

Validity is hereditary and, by Lemma 3.3(ii), a property of the block alone: no view is consulted. (W1) is the creator-justification rule and (W2) excludes justification regression of the signer's own blocks.

Lemma 3.6 (Own chain)**.** *Let m be valid and signed by v, and write $C(m)$ for the sequence*

$$C(m) = (m,\ \mathrm{self}(m),\ \mathrm{self}^2(m),\ \dots),$$

which ends at α_v. Then the blocks by v that are $\leq m$ are exactly the elements of $C(m)$ other than α_v.

Proof. By well-founded induction on m. By (W2) the blocks by v in $J^*(m)$ are $\mathrm{self}(m)$ and those in $J^*(\mathrm{self}(m))$; by (W1) and the hypothesis, the latter are the blocks of $C(\mathrm{self}(m))$. □

Define the *height* of a valid block as $\mathrm{ht}(m) = |C(m)| - 1$, its position on its signer's chain.

3.3 Equivocation

Definition 3.7 (Equivocation)**.** Validator v *equivocates* in σ if there are distinct $m_1, m_2 \in \sigma_v$ with $m_1 \not\leq m_2$ and $m_2 \not\leq m_1$. Write $\mathcal{E}(\sigma)$ for the set of equivocators and $F(\sigma) = w(\mathcal{E}(\sigma))$ for the *fault weight*. For $v \notin \mathcal{E}(\sigma)$ with $\sigma_v \neq \emptyset$, σ_v is a chain and its maximum is the *latest message* $\ell_v(\sigma)$.

Since equivocation is a property of pairs, $\mathcal{E}$ and F are monotone: if $\sigma \subseteq \sigma'$ then $F(\sigma) \leq F(\sigma')$.

Proposition 3.8 (Three criteria agree)**.** *Let σ be a protocol state of valid blocks and $v \in V$. The following are equivalent.*

(a) v equivocates in σ (incomparability, the CBC criterion).

(b) There are distinct $m_1, m_2 \in \sigma_v$ with $\mathrm{self}(m_1) = \mathrm{self}(m_2)$ (siblings: the slot criterion of Section 6).

(c) There are distinct $m_1, m_2 \in \sigma_v$ with $\mathrm{ht}(m_1) = \mathrm{ht}(m_2)$ (the sequence-number criterion of the implementation).

Proof. (b)⇒(c): siblings have equal height. (c)⇒(a): if $m_1 \leq m_2$ then by Lemma 3.6 $m_1 \in C(m_2)$, and distinct elements of a chain have distinct heights. (a)⇒(b): by Lemma 3.6 the sets $C(m_1)$ and $C(m_2)$ are chains ending at α_v; let c be their greatest common element. Neither m_i is c, since $m_i \leq m_j$ is excluded, so each chain has an element c_i immediately above c, with $\mathrm{self}(c_i) = c$. If $c_1 = c_2$ it would be a greater common element, so $c_1 \neq c_2$. Both lie in σ, which is closed under J. □

Remark 3.9. The Rocq development of the slashing subsystem defines equivocation by criterion (c) (`equivocates` in `DAGState.v`). The proposition is the bridge from that definition to the one CBC's safety argument uses, and it holds only for valid blocks: (W2) is what makes (c) imply (a).

4 Validators as frames

4.1 One program, many frames

Each validator runs in its own frame. An honest validator is the rotation $H_v = v \cdot H$ of a single program H, written in the identity frame and given in Section 5. Every literal name in H is rotated, so H_v has its own channels, its own block structure, and its own datum v on the blocks it signs. Received names are not rotated: a rotated receive binds in its own frame and a rotated use undoes it, so a name that H_v receives and later embeds in a block appears there exactly as it arrived. This is why justifications are intact in Definition 3.1.

Reading foreign frames. A block by u has its structure in frame u, and H must read blocks from every frame. It does so with a *decoder bank*, one decoder per relative frame:

$$Dec \;=\; \prod_{g \in G} \texttt{for}(@\langle slf_g!(@\langle X_p, \delta_p\rangle) \mid est_g!(@\langle X_E, \delta_E\rangle) \mid X_R, g\rangle \Leftarrow in)\, Accept$$

The literal datum g in each pattern selects the decoder whose frame the block is written in. The bank is invariant under rotation: rotating it by v re-indexes $g \mapsto vg$. So H_v reads every frame, and the rotated decoder that fires binds the same variables as the unrotated one would. We take $G = V$ finite in what follows; for $V \subsetneq G$ the bank ranges over V and the product below is over a subset rather than a subgroup.

4.2 Signatures as a frame discipline

A Byzantine validator b runs an arbitrary process Z_b, subject to one restriction.

Definition 4.1 (Signature discipline)**.** Z_b is *disciplined* if every block it constructs, as opposed to forwarding a name it received, has the form $\lfloor b;\ p,\ E,\ J \rfloor$.

A disciplined process may withhold, delay, send different blocks to different validators, equivocate, and construct invalid blocks; it cannot sign for anyone else. In frame terms, it speaks only in its own frame. The discipline is the model's stand-in for unforgeability of signatures, and it is the only assumption made about Byzantine behaviour.

Remark 4.2 (Chaos is not a Byzantine validator). The constant $\mathbf{1}$ of rho_G represents all behaviours, and it would be natural to model a Byzantine validator by it. That fails: $\mathbf{1} \mid P \equiv \mathbf{1}$, so a chaotic component absorbs the honest ones. What a Byzantine validator has is chaos *confined*

to a frame: every disciplined behaviour, and nothing else. Whether that confinement can be expressed as an operator on $\mathbf{1}$, say an orbit-restricted chaos $\mathbf{1}_b$, is left open.

4.3 The system

With $B \subseteq V$ the Byzantine validators, the system is

$$Sys_B \;=\; \prod_{v \in V \setminus B} v \cdot H \;\mid\; \prod_{b \in B} Z_b, \qquad Sys_\emptyset \;=\; \prod_V H,$$

so the all-honest system is an orbit product, and the validator set is a twisted root of the network with trivial stabilisers: every validator runs the same program in its own frame, which is sharding in the sense of [2, §6].

Lemma 4.3 (Frame-faithful channels)**.** *Every channel in a configuration reachable from Sys_B whose receivers belong to honest validators has the form $v \cdot c$ for a base channel c whose code mentions exactly one name at its outermost level, with datum e. Two such channels share a code only if they are equal. Hence every communication with an honest receiver is between equal names, its coefficients are the identity, and it reduces as in the rho calculus without data.*

Proof. The code of $v \cdot c$ determines v as the datum of its outermost name, and the action is free on names. Equality of names then gives both coefficients e, and the reduct $P\sigma + P\sigma$ is $P\sigma$ by idempotence (Proposition 3.6 of [2]). □

So the cancellation behaviour of the oriented communication rule never fires in the model. Byzantine processes could address isotropic channels of their own and communicate among themselves with non-trivial coefficients, but no honest process listens there.

Proposition 4.4 (Relabelling symmetry)**.** *Suppose w is constant and* Est *commutes with relabelling, that is,* $\mathrm{Est}(g \cdot \sigma) = g \cdot \mathrm{Est}(\sigma)$*. Then $g \cdot Sys_\emptyset \equiv Sys_\emptyset$ for every $g \in G$, every run of $Sys_\emptyset$ rotated by g is a run, and the state space may be explored modulo G.*

Proof. Orbit products are invariant, and Theorem 3.4 and Theorem 5.1 of [2] give the run correspondence and the quotient. □

Remark 4.5 (Tie-breaking is a symmetry breaker). The hypothesis on Est fails for every estimator that breaks ties by validator identity or by hash order, which includes the implementation's fork choice. This is not a defect. By the argument of Lehmann and Rabin [1], a symmetric deterministic protocol started in a symmetric state can be held in symmetric states by a scheduler that moves every copy in turn; with equal weights and a balanced initial split, such a scheduler could keep binary consensus from ever deciding. The tie-break is the symmetry breaker, and the proposition says exactly where it is needed: nowhere except at ties.

5 The honest validator

5.1 Terms

The program H keeps its view S, a protocol state, and its own latest block ℓ on a private linear channel st, so its local steps are serialised. It has an inbox in, a slot base name sl, an evidence

channel *ev* and a decision channel *fin*. With A the set of anchors,

$$\begin{aligned}
H &= st!(A, \alpha_e) \mid Dec \mid Prop \mid Fin \\
Accept(m) &= \texttt{for}(S, \ell \leftarrow st)\ \texttt{let}\ (S', N) = \mathrm{merge}(S, m)\ \texttt{in} \\
&\quad \big(st!(S', \ell) \mid \textstyle\prod_{n \in N} Watch(n)\big) \\
Prop &= \texttt{for}(S, \ell \Leftarrow st)\ \texttt{let}\ m = \lfloor e;\ \ell,\ \mathrm{Est}^\circ(S),\ \mathrm{tips}(S) \rfloor\ \texttt{in} \\
&\quad \big(st!(S \cup \{m\}, m) \mid \textstyle\prod_{g \in V} in_g!(m) \mid Watch(m)\big) \\
Watch(n) &= \mathrm{slot}(n)!(n) \mid \texttt{for}(x \leftarrow \mathrm{slot}(n)\ \&\ y \leftarrow \mathrm{slot}(n))\, ev!(x, y) \\
\mathrm{slot}(n) &= @\langle sl!(\mathrm{self}(n)), e \rangle \\
Fin &= \texttt{for}(S, \ell \Leftarrow st)\ \big(st!(S, \ell) \mid \textstyle\prod_{p \in \mathrm{oracle}(S)} fin!(p)\big)
\end{aligned}$$

Here $Accept(m)$ is the continuation of the decoder that received m, and Est° picks an element of Est. The local functions are:

- $\mathrm{merge}(S, m)$: if m is valid, the new blocks are $N = (J^*(m) \cup \{m\}) \setminus S$ and $S' = S \cup N$; otherwise $N = \emptyset$ and $S' = S$. Because m carries its own closure, no buffering of missing dependencies is needed.
- $\mathrm{tips}(S)$: the maximal elements of S under $\leq$, so that $J^*(m) \cup A = S$ for the proposed block and $\mathrm{Est}(J^*(m) \cup A) = \mathrm{Est}(S)$.
- $\mathrm{oracle}(S)$: the properties that the clique oracle of Section 8 certifies on S.

The three persistent receives on *st* race with every acceptance, so H may propose or decide between any two deliveries, which is all the nondeterminism of timing that the protocol admits. The channel *st* holds exactly one message at all times, which serialises the local steps.

5.2 Observers

At validator o the slot is $o \cdot \mathrm{slot}(n) = @\langle sl_o!(\mathrm{self}(n)), o \rangle$: its structure is in o's frame, its content is n's self-justification, intact. Different observers therefore have disjoint slot namespaces, and each observer detects in its own view. By (W1) the self-justification of a block determines its signer, so a slot belongs to one validator.

6 Equivocation as a join

Theorem 6.1 (Detection)**.** *Let C be reachable from Sys_B, let o be honest with view S in C, and let $v \in V$.*

(i) Soundness. *If o has emitted $ev_o!(x, y)$ then $x \neq y$, $\mathrm{sdr}(x) = \mathrm{sdr}(y)$, and $\mathrm{sdr}(x)$ equivocates in S.*

(ii) Completeness. *If v equivocates in S, then in every continuation of C in which o's detector joins are weakly fair, o eventually emits $ev_o!(x, y)$ with $\mathrm{sdr}(x) = v$.*

Proof. Fix a slot s of o. The only senders on s are the *Watch* terms of o, and the only receivers on s are their joins (Lemma 4.3: no other channel shares its code). Since merge adds each block

to the view at most once and *Prop* adds a fresh block, o executes $Watch(n)$ exactly once for each block $n \in S$ with $\mathrm{slot}(n) = s$. So, writing k for the number of such blocks and f for the number of joins on s that have fired, s carries $k - 2f$ announcements and $k - f$ joins, and every announcement ever made on s is a distinct block of S whose self-justification is the content of s.

(i) A firing join consumes two announcements, hence two distinct blocks of S that are siblings; by (W1) they have one signer, and by Proposition 3.8, (b)⇒(a), it equivocates.

(ii) By Proposition 3.8, (a)⇒(b), S contains two distinct siblings by v, so some slot s of v has $k \geq 2$. If no join on s ever fires then s carries $k \geq 2$ announcements and k joins forever; a join is then continuously enabled, and weak fairness makes one fire. □

What the pointer buys. The slot is a name that o builds from one block, without looking at any other. That is possible only because the block carries its self-justification: without the pointer an observer could not name the place where a rival would appear, and detecting a rival would require comparing blocks pairwise. With the pointer the comparison is done by the gate of the communication rule, which compares codes. This is the comparator of the decoder in [2, §6]: building a channel name from content turns equality into rendezvous.

Evidence is checkable. The pair (x, y) emitted on *ev* is two signed blocks with the same self-justification. Any third party can check it from the blocks alone, which is what accountability requires and what the amended slashing specification of the implementation asks of slashable evidence.

Duplicates. A Byzantine relay can deliver the same block to o many times. merge discards repeats, so no announcement is duplicated, and a block can never be mistaken for its own rival. The one negative test in the model, $n \notin S$, lives in local computation, not in communication.

7 Safety

7.1 Views

Write $\mathrm{view}_o(C)$ for the view of honest o in configuration C, and $\Gamma(C)$ for the set of all blocks occurring in C.

Lemma 7.1 (Views)**.** *In every configuration reachable from Sys_B:*

(i) each honest view is a protocol state of valid blocks, and views only grow;

(ii) the blocks of an honest v in $\Gamma(C)$ form a single chain, and ℓ is its maximum;

(iii) for every set $\sigma \subseteq \Gamma(C)$, $\mathcal{E}(\sigma) \subseteq B$, so $F(\sigma) \leq w(B)$.

Proof. (i) merge adds a valid block together with its closure. A proposed block m is valid: its self-justification is ℓ, its own latest block, which is in S; the blocks of v in $J^*(m) = S \setminus A$ are those of the chain below ℓ by (ii); and $\mathrm{est}(m) \in \mathrm{Est}(S) = \mathrm{Est}(J^*(m) \cup A)$. (ii) Proposals are serialised on *st* and each takes the previous ℓ as its self-justification; by discipline no other process signs for v. (iii) By (ii), two blocks of an honest validator are comparable. □

7.2 Consensus safety

Definition 7.2 (Estimate safety)**.** Fix a threshold t. A property p of estimates is *t-safe* at a protocol state σ of valid blocks if every protocol state $\sigma' \supseteq \sigma$ of valid blocks with $F(\sigma') \leq t$ has $\mathrm{Est}(\sigma') \models p$, meaning every element satisfies p.

Theorem 7.3 (Consensus safety)**.** *Let C be reachable from Sys_B with $w(B) \leq t$. Let honest o_1 and o_2 hold views σ_1, σ_2 at any two configurations of the run up to C, and let p_1 be t-safe at σ_1 and p_2 t-safe at σ_2. Then $p_1 \wedge p_2$ is satisfiable.*

Proof. $\sigma = \sigma_1 \cup \sigma_2$ is a protocol state of valid blocks by Lemma 7.1(i), contains both, and is a subset of $\Gamma(C)$, so $F(\sigma) \leq w(B) \leq t$ by Lemma 7.1(iii). Every element of $\mathrm{Est}(\sigma)$ satisfies both. □

The theorem is CBC's, and the proof is CBC's. What the model contributes is the three facts it rests on, each of which is now a statement about terms: that unions of views are protocol states, because blocks are quoted with their closures; that fault weight in a union is bounded by Byzantine weight, because honest validators are serialised in their own frames; and that an observer can compute fault weight, by Theorem 6.1.

Proposition 7.4 (Futures are reachable)**.** *Let o_1, o_2 be honest with views σ_1, σ_2 in C. If o_2 proposes in some continuation, then in every continuation that extends that one and delivers every message, o_1's view comes to contain $\sigma_1 \cup \sigma_2$.*

Proof. o_2's next block has closure $\supseteq \sigma_2 \setminus A$ and is sent to in_{o_1}; it is valid, so merge adds its closure. □

So the set-theoretic futures of CBC are not only a convenient over-approximation: the union that the safety proof uses is a view that an honest validator actually reaches.

8 The clique oracle

Theorem 7.3 needs an observer to *know* that p is safe. The clique oracle is the standard sufficient condition. We state it for any estimator with the following property.

8.1 Majoritarian estimators

Definition 8.1. Fix a property p. A block m *supports* p if $\mathrm{est}(m) \models p$. The estimator is *$p$-majoritarian* if, for every protocol state σ of valid blocks, each latest message of a non-equivocator can be classified as supporting, opposing or neutral for p, supporting messages support p, and

$$w\{v : \ell_v(\sigma) \text{ supports}\} \;>\; w\{v : \ell_v(\sigma) \text{ opposes}\} \quad\Longrightarrow\quad \mathrm{Est}(\sigma) \models p.$$

Example 8.2. Binary consensus with weighted majority of latest messages, and either value on ties, is p-majoritarian for $p = (\text{estimate} = 1)$ with no neutral messages.

Lemma 8.3 (LMD-GHOST)**.** *Let estimates be blocks of a tree of main parents, and let* Est *be the greedy heaviest-observed-subtree rule, with each latest message's weight flowing down its main-parent spine. For a block T, let p_T be T* is on the spine of the estimate*; classify a latest message as supporting if T is on its spine, neutral if its block is a proper ancestor of T, and opposing otherwise. Then* Est *is p_T-majoritarian.*

Proof. Walk the spine of T from the root. At each fork on it, the child towards T carries the weight of every supporting message and of no opposing one, while the rival children together carry at most the opposing weight; neutral messages count on neither side. So the child towards T is strictly heavier than each rival, and has positive score, and the greedy walk takes it. It therefore reaches T. □

8.2 The theorem

Definition 8.4 (Clique)**.** Let σ be a protocol state of valid blocks. For non-equivocators a, b in σ write n_{ab} for the latest block by b that is $\leq \ell_a(\sigma)$, when it exists. A set Q of non-equivocators is a *p-clique* in σ if for every $a \in Q$, ℓ_a supports p, and for all $a \neq b$ in Q, n_{ab} exists and

$$\text{every block } n \in \sigma_b \text{ with } n_{ab} \leq n \text{ supports } p. \tag{†}$$

Condition (†) ranges over the segment of b's chain from a's view of b up to b's latest message, *both ends included*.

Theorem 8.5 (Clique oracle)**.** *Let* Est *be p-majoritarian, let Q be a p-clique in σ with $q = w(Q)$, and let $\sigma' \supseteq \sigma$ be a protocol state of valid blocks with $F(\sigma') < 2q - W$. Then* $\mathrm{Est}(\sigma') \models p$*. Hence p is t-safe at σ for every $t < 2q - W$.*

Proof. Let $Q' \subseteq Q$ be the members that do not equivocate in σ', so $w(Q') \geq q - F(\sigma')$. Call a block of $\sigma' \setminus \sigma$ by a member of Q' a *defector* if it does not support p.

No defectors. Suppose not, and take a defector m by $a \in Q'$ minimal for $\leq$ (Lemma 3.3). Since a's blocks form a chain and $m \notin \sigma$, $\ell_a(\sigma) \leq m$, so for every $b \in Q' \setminus \{a\}$, $n_{ab} \leq m$. Consider $\tau = J^*(m) \cup A$ and $b \in Q'$, and let n be b's latest block in τ; it exists, since $\ell_a(\sigma)$ or n_{ab} is in τ, and $n \geq n_{ab}$ (or $n \geq \ell_a(\sigma)$ when $b = a$). If $n \in \sigma$, it supports p by (†), or because it is $\ell_a(\sigma)$. If $n \notin \sigma$, it is not a defector, by minimality of m, so it supports p. Thus in τ the supporting weight is at least $w(Q') \geq q - F(\sigma')$, while opposing messages come from validators outside Q, of weight at most $W - q$. Since $q - F(\sigma') > W - q$ and Est is p-majoritarian, $\mathrm{Est}(\tau) \models p$; by validity (V), m supports p, a contradiction.

Conclusion. The latest message in σ' of each $b \in Q'$ is either in σ, where it is $\ell_b(\sigma)$ and supports, or in $\sigma' \setminus \sigma$, where it is not a defector. So supporting weight in σ' is at least $q - F(\sigma') > W - q$, which bounds the opposing weight. □

The condition $F(\sigma') < 2q - W$ is, divided by W, the implementation's fault-tolerance figure: certifying at $\mathrm{FT} = (2q - W)/W \geq \theta$ tolerates any equivocating weight below θW.

Corollary 8.6 (Finality safety)**.** *Let C be reachable, and let honest o_1 and o_2, at views σ_1 and σ_2 in the run up to C, certify p_1 and p_2 by p_i-cliques of weight q_i under p_i-majoritarian estimators. If $p_1 \wedge p_2$ is unsatisfiable, then $w(B) \geq F(\sigma_1 \cup \sigma_2) \geq \min_i(2q_i - W)$, and the detectors of any honest validator whose view contains $\sigma_1 \cup \sigma_2$ name equivocators of that much weight.*

Proof. If $F(\sigma_1 \cup \sigma_2) < \min_i(2q_i - W)$, Theorem 8.5 applied twice gives $\mathrm{Est}(\sigma_1 \cup \sigma_2) \models p_1 \wedge p_2$. The rest is Lemma 7.1(iii), Theorem 6.1 and Proposition 7.4. □

So conflicting finality costs at least θW of stake that is caught equivocating: accountable safety in the sense of [4], with the accounting done by joins.

9 The implementation

9.1 Crosswalk

Table 1 relates the model to `f1r3node-rust`.

model	`f1r3node-rust`
block name $\lfloor v; p, E, J \rfloor$	`BlockMessage`; the hash replaced by the quoted block
self(m), (W1)	creator justification; `Validate::sequence_number`
(W2)	`justification_regressions`, `justification_follows`
ht(m)	`seq_num`
Proposition 3.8	bridges `equivocates` (Rocq, criterion (c)) to CBC's criterion (a)
slot join, Theorem 6.1	`check_equivocations`: creator justification $\neq$ latest message of creator means the slot is occupied
merge with closure	block retrieval plus `CasperBuffer`
Est, Lemma 8.3	`Estimator` (LMD-GHOST)
Definition 8.4, Theorem 8.5	`CliqueOracle`, `never_eventually_see_disagreement`

Table 1: Correspondence between the model and the node.

Two differences are by design. First, the implementation sends hashes and fetches missing dependencies; in the model a name is its content, and the dependency buffer disappears. Restoring hash pointers would replace merge by a join that waits on a persistent *seen* fact for each justification, which is exactly what the buffer does. Second, the implementation's simple equivocation check compares a new block's creator justification with the receiver's latest message of the creator. That flags the second occupant of a slot, and also a block whose slot is older than the creator's latest; both are incomparable pairs, so by Proposition 3.8 it detects the same validators as the slot join.

9.2 The oracle omits a hypothesis

The node's agreement graph puts an edge between a and b when both latest messages have the target T on their main-parent spine and, in each direction, a walk down b's self-justification chain from ℓ_b towards $lm_{a\to b}$, a's view of b, finds no disagreeing block. Compared with $(\dagger)$, the walk differs in two places.

1. It starts at self(ℓ_b) and stops at the stopper self($lm_{a\to b}$), or at $lm_{a\to b}$ itself when that block has no self-justification. In the second case $lm_{a\to b} = n_{ab}$ is never examined.

2. A visited block below T's height that lies on T's own spine is counted as not disagreeing. In the terms of Lemma 8.3 it is neutral, and $(\dagger)$ requires support.

Either way, nothing checks that n_{ab} supports T. The minimality step of Theorem 8.5 uses exactly that: a defector's justification must contain a supporting view of every clique member. When a's view of b predates T, b's weight does not reach T in a's later views, and the clique's weight is not there to hold a.

9.3 A counterexample

Take validators a, c and $b_0, \ldots, b_{k-1}$, with $w(a) = w(b_i) = 1$ and $w(c) = 1.01$, so $W = k + 2.01$. No validator ever equivocates.

1. Each b_i proposes its first block b_i^1 on b_{i-1}^1, seeing all earlier ones. Then a proposes T on b_{k-1}^1, and c, not seeing T, proposes T' on b_{k-1}^1. So T and T' are siblings.

2. Each b_i receives T and proposes b_i^2 on T, before any other b_j^2 reaches it. Its view of a is T; its view of b_j is b_j^1.

3. An observer holding T and every b_i^2 runs the oracle on T. Agreeing weight is $k + 1$. Every pair is an edge: each $lm_{x \to b_i}$ is b_i^1, a first block, so the walk examines nothing. The clique is $\{a, b_0, \ldots, b_{k-1}\}$ and $\mathrm{FT} = (k - 0.01)/(k + 2.01)$.

4. Delivery of the b_i^2 to a and to one another is delayed. a receives T' and, at the fork under b_{k-1}^1, sees weight 1 on T (its own) against 1.01 on T': the b_i^1 are neutral there. It proposes a^3 on T'. Each b_i in turn receives a^3 and sees 1 on T against 2.01 on T', and proposes b_i^3 on a^3.

5. All messages are now delivered and every validator proposes on the head, which descends from T'. The oracle certifies T' with $\mathrm{FT} = 1$.

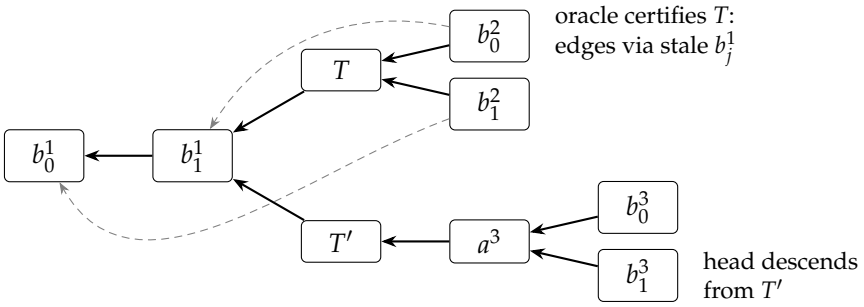


Figure 1: The schedule for $k = 2$. Solid arrows are main parents; dashed arrows are the stale views $lm_{b_i \to b_j} = b_j^1$ that the oracle accepts as edges.

A re-implementation of the node's edge rule and of LMD-GHOST, independent of the node and included with this note as `oracle_sim.py`, reproduces the schedule. With $k = 10$ it certifies T at FT ≈ 0.832, above the default threshold 0.67 (`defaults.conf`, with the synchrony constraint threshold at its default of 0), and then certifies the sibling T' at FT $= 1$. With $k = 3$ the figures are 0.597 and 1. Under the rule (†) the first certificate has no edges, and the oracle's figure for T is negative. Equivocating weight is zero throughout, so this violates Corollary 8.6, not only its accountable form: two honest observers finalize incompatible blocks and there is nobody to slash.

Remark 9.1 (Scope of the claim). The schedule is checked against a re-implementation, not against the node. It rests on three readings of `safety/clique_oracle.rs`:

- no test of $lm_{a \to b}$ against the target when edges are computed;

- the stopper rule and the two-sided height test in the disagreement walk;
- agreement as main-spine membership.

A regression test in the node's finalized-floor suite that builds this DAG is the next step. The Rocq development is consistent with the finding: its capstones are disclosed as proving determinism of the floor, not CBC finalization safety, and its accountable-safety theorem takes the incompatibility hypothesis as an assumption over a single snapshot.

9.4 Repair

Require (†): in each direction, also check that $lm_{a\to b}$ has the target on its main-parent spine, and treat every visited block that does not as disagreeing, whatever its height. In code, the walk must include its lower endpoint, and the below-height branch must return disagreement for blocks on the target's own spine.

This undoes the relaxation that was introduced to cure a finality stall, in which a stale window in one validator's view held certification back. Under Theorem 8.5 that stall was the oracle being right: a validator whose view of b predates T is not yet held by b's weight, and certification must wait until its view is refreshed. A positive synchrony constraint threshold forces such refreshes and shortens the wait; it does not replace the check, since it is a configuration choice and its default is 0.

10 What the group contributes

The group action earns its place in three ways, and it is worth being plain about where it does not.

- *Signer as datum.* The name of a block records who signed it, and the frame of its structure agrees with the datum. Reading a block from any validator is one decoder bank, invariant under rotation.
- *One program.* The honest network is an orbit product of a single program, and the protocol's symmetry under relabelling of validators is a theorem of the calculus (Proposition 4.4), with the symmetry-reduction quotient of [2, §5] available to a model checker. It also locates the one place where symmetry must be broken.
- *Signatures as frames.* Unforgeability becomes a syntactic discipline: a validator speaks only in its own frame.

The cancellation behaviour of the oriented communication rule plays no part: by Lemma 4.3 every communication with an honest receiver is between equal names. Neither do chaos and summation. A model in the plain rho calculus with a signer field on names would prove the same safety theorems. What it would lose is the statement that the validators are one program.

11 Further work

- *Mechanisation.* Sections 3, 7 and 8 are first-order and finitary, and fit the existing Rocq developments. Theorem 8.5 is the theorem the finalized-floor dossier lists as out of scope.

- *Multi-parent merging.* Lemma 8.3 uses main-parent spines and is indifferent to secondary parents. Whether finality of a block on the spine gives finality of the effects merged into it is a separate question about the merge, which the model does not reach.
- *Identity-frame presentation.* The decoder bank reads foreign frames by enumerating G. Frame-fixing with dynamic decorations, $[\delta^{-1}]X$ in the notation of [2, §6], would read every frame with one receive, at the price of the word problem for G, which is trivial for finite G.
- *Liveness.* Nothing here is about liveness, which in an asynchronous setting requires the synchrony assumptions that the model deliberately leaves out.
- *Hash pointers.* Replace quoted closures by names that refer to blocks without containing them, with a persistent *seen* fact per block and dependency waits as joins. Proposition 7.4 then needs a fetch protocol, and Lemma 3.3 needs collision resistance as a hypothesis.
- *Slashing.* Close the loop from *ev* to the bond map, so that the fault weight seen by the estimator falls when evidence is acted on.

Acknowledgements

This note was drafted in collaboration with Claude (Anthropic).

References

[1] D. Lehmann and M. O. Rabin. On the advantages of free choice: a symmetric and fully distributed solution to the dining philosophers problem. In *POPL*, pages 133–138. ACM, 1981.

[2] L. G. Meredith. Oriented processes: group-valued data, a chaos constant, and an equivariant COMM for the reflective higher-order calculus. F1R3FLY.io research note, draft 3, 2026.

[3] L. G. Meredith and M. Radestock. A reflective higher-order calculus. *Electronic Notes in Theoretical Computer Science*, 141(5):49–67, 2005.

[4] V. Zamfir, N. Rush, A. Asgaonkar, and G. Piliouras. Introducing the "minimal CBC Casper" family of consensus protocols. Ethereum research, 2018.

[5] V. Zamfir. Casper the friendly ghost: a correct-by-construction blockchain consensus protocol. Ethereum research, 2017.